NORTH SOUTH UNIVERSITY
Department of Electrical and Computer Engineering

CSE 115L — Programming Language I Lab

Laboratory Manual | Lab 09: Repetition — Counting Loops, `while` and `for`

Course faculty: **Mirza Mohammad Lutfe Elahi** | Lab instructor: **Moshiur Rahan**

Fall 2026 | Text: Hanly & Koffman, *Problem Solving and Program Design in C*, 8th edition | Follows Lecture 11, §5.1–§5.4 | 80 minutes

How to use this manual

This is the **ninth** of 21 laboratory sessions. Bring this manual, printed, and the **task sheet** issued with it — the task sheet is what the lab instructor signs, and it is worth marks. The lab follows the same steps every week: a **concept recap** (the syntax you need at the keyboard, not a repeat of the lecture), **worked examples** (every program here has been compiled and run, and every line of output is the real output), **guided tasks** with blanks to complete, **in-lab exercises** graded ***Easy***, ***Mediocre*** and ***Challenging***, and **home tasks** for before the next lab.

Lab performance marks are awarded in the room. Show your working program to the lab instructor before you leave; nothing is accepted afterwards. Tasks marked with **N** use digits **4 to 7** of your own student ID — the four that are unique to you — so your output will differ from your neighbour's.

This is the lab where programs start to repeat themselves. Everything you have written so far ran each statement at most once. From today a handful of lines can do work a thousand times — and a loop that is wrong by one is the most common defect in this course.

Two warnings before you start. An infinite loop locks the console: press **Ctrl + C** in the program window, or close it, to get back. And **keep warnings on** — *Settings* → *Compiler* → *Compiler settings*, tick `-Wall`. Nothing in C9.2 stops the build; two of its four defects are reported only because `-Wall` is on, and the other two are reported by nothing at all.

Lab	Follows	Topic	Sections
09	Lecture 11	Repetition in programs; the counting loop and `while`; the loop control variable; accumulating a sum and a product; the `for` statement; increment, decrement and compound assignment	§5.1–§5.4

Lab 09 | Repetition — Counting Loops, `while` and `for`

Follows Lecture 11 · Hanly & Koffman §5.1–§5.4 **Duration: 80 minutes**

Learning Objectives

1. Name the three things every counting loop must do, and point at each one in the code.
2. Write a `while` loop that repeats a known number of times.
3. Accumulate a **sum** and a **product**, and say what each accumulator must start at and why.
4. Write the same loop as a `for`, and convert in both directions.
5. Use `++`, `--` and the compound assignment operators correctly.
6. Diagnose an **off-by-one** loop and an **infinite** loop from their output alone.

Minutes	Activity
0–10	Concept recap (§1); the three parts of a loop go on the board.
10–24	Instructor demonstrates Worked Examples 9.1, 9.2 and 9.3.
24–34	Guided Tasks G9.1 and G9.2.
34–72	In-lab exercises, worked in tier order. Instructor circulates and signs off.
72–80	Task sheet signed off; briefing on the home tasks.

Where this lab sits

This is the practice session for **Lecture 11** — §5.1 to §5.4, the first third of Chapter 5. Your homework is the Worksheet 5 questions on repetition, counting loops, the `while` statement, sum-and-product accumulation and the `for` statement.

One loop form at a time. Today's loops all repeat a number of times you know before the loop starts. Loops that stop when the *data* says so — sentinel values, `do-while`, flags — and loops inside loops are **Lab 10**. Do not use them here; every task below is solvable with `while` or `for` alone.

Lab Quiz 2 is next week, at the start of Lab 10. It is 40 minutes, closed book, and covers **Labs 05–08** — functions with arguments and results, and the whole of Chapter 4. Revise from §5 of the Lab 06, 07 and 08 manuals.

1 Concept Recap

1.1 What repetition buys you

[§5.1 · Table 5.1]

Three lines that run five times beat fifteen lines that run once: the program is shorter, and changing `5` to `500` costs one character. A **loop** is a statement that repeats; the statements inside it are the **loop body**.

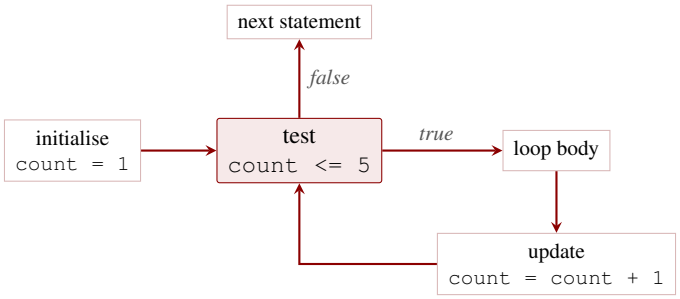
Kind of loop	It stops when	Where you meet it
Counting	a count reaches a limit known in advance	**this lab** — `while`, `for`
Sentinel-controlled	a special data value arrives	Lab 10
Flag-controlled	a condition inside the body becomes true	Lab 10

1.2 The three jobs of a counting loop

[§5.2 · Figure 5.2]

Every counting loop does exactly three things to its **loop control variable**. Miss one and the loop is wrong in a way you can predict:

Job	In the code	Leave it out and
Initialise	`int count = 1;`	the loop tests a garbage value
Test	`while (count <= 5)`	there is no loop
Update	`count = count + 1;`	**it never stops**



The test is at the top*: a counting loop whose condition is false at the start runs its body* ***zero*** *times. That is not a bug — it is the point.*

1.3 Accumulating a sum and a product

[§5.3 · Figure 5.5]

An **accumulator** is a variable that collects a result one pass at a time. It must be initialised *before* the loop, to the value that leaves the answer unchanged:

```
int    sum     = 0;     /* 0 added to anything changes nothing     */
int    product = 1;     /* 1 multiplied by anything changes nothing */
double total   = 0.0;
```

A `sum` that starts at `1` is one too big. A `product` that starts at `0` is **always zero**, whatever the data — and nothing warns you.

1.4 The `for` statement

[§5.4 · Figure 5.7]

`for` puts the three jobs on one line, in the order they happen:

```
for (initialise ; test ; update)
    statement;                    /* or a { } block */
```

These two loops are the same loop:

As a `while`	As a `for`

`int i = 1;`
`while (i <= 5) {`
`  printf("%d\n", i);`
`  i = i + 1;`
`}`
	`int i;`
	`for (i = 1; i <= 5; i++) {`
	`  printf("%d\n", i);`
	`}`

Use `for` when you know the number of repetitions, and `while` when you do not. Both work either way; the choice is about telling your reader which situation they are in.

1.5 ++, − and compound assignment

[§5.4 · Table 5.6]

Written	Means	Note
`i++`, `++i`	`i = i + 1`	as a *statement* on its own, identical
`i−`, `−i`	`i = i − 1`	as above
`sum += x`	`sum = sum + x`	also `−=`, `*=`, `/=`, `%=`
`j = i++`	use `i`, *then* add 1	`j` gets the **old** `i`
`j = ++i`	add 1, *then* use `i`	`j` gets the **new** `i`

The last two rows are the reason this course writes `i++` only as a complete statement, or in the update part of a `for`. Mixing `++` into a larger expression is legal, hard to read, and a standing invitation to a defect.

2 Worked Examples

Worked Example 9.1 — A counting loop with `while`

[§5.2]

Five rows of a conversion table, and the value the control variable is left holding.

```
#include <stdio.h>

int main(void)
{
    int count = 1;

    printf("Inches        cm\n");
    while (count <= 5) {
        printf("%6d  %8.2f\n", count, count * 2.54);
        count = count + 1;
    }

    printf("Done. count is now %d\n", count);
    return 0;
}
```

```
Inches        cm
     1      2.54
     2      5.08
     3      7.62
     4     10.16
     5     12.70
Done. count is now 6
```

`count` ends at **6**, not 5. The loop stops when the test fails, which means the control variable has already gone one past the last value it was used for. Expecting 5 there is the off-by-one error in its purest form.

Worked Example 9.2 — Accumulating a total

[§5.3]

Five marks in, one total and one average out. The user types `72 85 64 91 78`.

```c
#include <stdio.h>

int main(void)
{
    int i = 1, mark, total = 0;

    while (i <= 5) {
        printf("Mark %d: ", i);
        scanf("%d", &mark);
        total = total + mark;
        i = i + 1;
    }

    printf("Total   %6d\n", total);
    printf("Average %9.2f\n", (double) total / 5);
    return 0;
}
```

```
Mark 1: 72
Mark 2: 85
Mark 3: 64
Mark 4: 91
Mark 5: 78
Total      390
Average     78.00
```

Two things to notice. `total` is initialised to `0` *outside* the loop — put that line inside and every pass throws the previous marks away. And the cast in `(double) total / 5` is the Lab 03 lesson again: without it, `390 / 5` is integer division, and a total of `388` would average to `77.00` instead of `77.60`.

Worked Example 9.3 — `for`, with two accumulators

[§5.4]

```c
#include <stdio.h>

int main(void)
{
    int sum = 0, product = 1, i;

    for (i = 1; i <= 6; i++) {
        sum += i;
        product *= i;
    }

    printf("sum     = %d\n", sum);
    printf("product = %d\n", product);
    return 0;
}
```

```
sum     = 21
product = 720
```

One loop, two accumulators, two different starting values. Change `product`'s initialisation to `0` and the program still compiles, still runs, and prints `product = 0` — a wrong answer delivered with complete confidence.

3 Guided Tasks — Complete the Code

Not marked. Do them anyway — the exercises assume you have.

Working space

G9.1 Complete the counting loop

This loop must print the squares of 1 to 10 and then the number of lines it printed. Fill the five blanks.

```
int i, lines = _______(1) ;

i = _______(2) ;
while (i ________(3) 10) {
    printf("%3d %5d\n", i, i * i);
    lines ________(4) 1;
    _________(5) ;
}
printf("%d lines printed.\n", lines);
```

Then say what happens if blank 5 is left empty, and what happens if blank 3 is < instead.

Blank 5 empty →	*Blank 3 as < →*

G9.2 The same loop, three ways

Fill the empty cells so that every row describes the same *repetition.*

As a `for` header	As `while`, with its init and update	Times the body runs
`for (i = 1; i <= 7; i++)`		
	`i = 0; while (i < 10) ...i++;`	
`for (i = 10; i >= 1; i-)`		
`for (i = 2; i <= 2; i++)`		
`for (i = 5; i <= 1; i++)`		

4 In-Lab Exercises

Your personal data value

N is the four digits in positions 4 to 7 of your own student ID. If your ID is 2411234, then N = 1234.

• *Easy*

E9.1 Count down, then count the counting

Using a `while` loop, print the integers from 10 down to 1 on one line each, then print how many numbers you printed and the value the control variable holds after *the loop. Both printed values must come from variables, not from typing `10` and `11`.*

E9.2 The same thing as a `for`

Rewrite E9.1 as a single `for` loop with nothing in the body but the `printf`. Both programs must produce byte-for-byte identical output. Keep both files — the task sheet asks the instructor to see them side by side.

• *Mediocre*

M9.1 Sum, product and average in one pass

*Read **five** integers in a counting loop and print their sum, their product and their average to two decimal places. Initialise each accumulator correctly, and use compound assignment (`+=`, `*=`) rather than the long form. Then run it again with one input changed to `0` and say, in one line, why only one of the three results collapses.*

M9.2 A table driven by N

*Print a table of the first **eight** multiples of **N**, as `k`, `N * k` and `N * k` as a percentage of `N * 8`, in three aligned columns with a heading row. Use `%nd` and `%n.mf` field widths so the columns line up for* any *four-digit N — test with N and with `9999`.*

M9.3 Find the off-by-one

This loop is meant to add the integers 1 to `n` inclusive. Type it, run it with `n = 5`, and record what it prints. Then make the smallest possible change that fixes it, and say which of the loop's three jobs was wrong.

```c
int i, n = 5, sum = 0;
for (i = 1; i < n; i++)
    sum += i;
printf("Sum of 1 to %d is %d\n", n, sum);
```

What it printed, the one-character fix, and which job was wrong:

• ***Challenging***

C9.1 Powers of two, and the moment they stop being right

Using one `for` loop and an `int` accumulator, print `2` raised to each power from 1 to 40, one per line, as `power` and `value`. Somewhere in that table the numbers stop making sense: find the first power where it happens, say what the value does, and explain it from the size of an `int`. Do not *use `pow` — the point is the accumulator. Then say how far the table gets correctly if the accumulator is a `double`, and what you lose in exchange.*

First wrong power, what the value does, and the `double` trade-off:

C9.2 Four defects in nine lines

***Nothing here stops the build** — it compiles, it runs, and it prints two numbers. **Two** of the four defects draw a warning under `-Wall`; the other **two** are reported by nothing at all and are visible only in the answer. Type it, build it with `-Wall` on, and fix one defect at a time, **rebuilding after each fix**.*

```c
#include <stdio.h>

int main(void)
{
    int    i, count, total;
    double average;

    count = 5;
    for (i = 1; i < count; i++);
        total = total + i;
    average = total / count;

    printf("Total   %d\n", total);
    printf("Average %.2f\n", average);
    return 0;
}
```

Hints, in no particular order: count the statements the `for` actually controls; count the passes; ask what `total` starts at; and ask what type the division on the `average` line is done in.

For each of the four: the defect, whether the compiler warned *or said* nothing, *and the corrected line. Then two more: with the first two defects fixed but the other two left alone, the program prints `Total 10` and `Average 2.00` — explain both numbers. And once all four are fixed, does the loop add `1+2+3+4` or `1+2+3+4+5`?*

5 Common Errors in This Lab

The ones that cost marks

1. **No update.** The control variable never changes, the test never fails, the program never ends. **No warning** — an infinite loop is legal C. **Ctrl + C** in the program window stops it.

2. **Off by one.** < where <= was meant, or starting at `0` when the first item is `1`. The loop runs, the answer is quietly wrong. Check it by hand with a limit of **1**, then **2**.
3. **An uninitialised accumulator.** `int total;` then `total += x` adds to whatever was in memory. Under `-Wall` gcc says `'total' is used uninitialized`; without `-Wall` it says nothing.
4. **`product = 0`.** The product of anything and zero is zero. No diagnostic, ever. A product accumulator starts at **1**.
5. **A semicolon after the loop header.** `for (i = 1; i <= 5; i++);` gives the loop an **empty body**; the indented line after it runs once. Indent it and gcc warns about misleading indentation — which is the only reason you will be told at all.
6. **= for == in the condition.** `while (i = 5)` assigns, yields 5, and loops forever. gcc warns: `suggest parentheses around assignment used as truth value`.
7. **Integer division in the average.** `total / count` with two `int`s truncates before the result reaches your `double`. Cast one side: `(double) total / count`.
8. **Declaring the control variable inside the loop body.** It is then a new variable each pass, and the loop cannot count. Declare it with the other variables at the top of `main`.

6 Home Tasks

1. **Worksheet 5** — the questions on repetition, counting loops and `while` (§5.1–§5.2), and on sum-and-product accumulation and `for` (§5.3–§5.4). These are your revision for **Assessment 3** and the **lab mid-term**.
2. Take the one-shot menu you wrote as E8.2 last week and put a counting loop around the *report*, not the menu: whichever option the user picks, print its line **N % 5 + 3** times. Making the *menu* repeat needs a conditional loop — that is next week.
3. Write the shortest program you can that loops forever without any statement inside the loop body. Then write the one-character change that makes it terminate. Bring both.
4. Hand-trace, on paper, `for (i = 0; i <= 4; i += 2) sum += i;` starting from `sum = 0`. Write the value of `i` and `sum` after every pass, and the value of `i` after the loop. Then run it and check.
5. Read ahead: §5.5, §5.7 and §5.8. **Lab Quiz 2 is at the start of Lab 10** — 40 minutes, closed book, Labs 05–08.

Notes